

Baitler — Project Documentation

Personal-assistant & data-organizer platform with AI/LLM capabilities layered on top. Generated 2026-05-30.

1. Overview

Baitler (a portmanteau of “Butler” + “AI”; **baitler.com**) is a personal-assistant and data-organizer application for managing a user’s files, data, and ideas, with AI/LLM capabilities layered on top. The **base portal** is the central hub where every feature comes together. Baitler is also exposed as an **MCP server**, making its knowledge base directly addressable by AI agents.

2. Architecture

A multi-surface monorepo with separate workspaces sharing a top-level `scripts/` directory for dev orchestration.

- **Frontend** — React 19 + Vite + TypeScript (strict), styled with TailwindCSS v4.
- **Backend** — a Rust HTTP API (Axum) with **SurrealDB** as the datastore.
- **Mobile** — native iOS and Android apps (planned, Phase 9).
- **scripts/** — runs frontend + backend together and builds both.

3. Technology Stack

Layer	Technologies
Frontend	React 19, TypeScript (strict), Vite, Tailwind v4, React Router 7, TanStack Query 5, Zustand, react-markdown + @tailwindcss/typography, TipTap (HTML editor)
Backend	Rust, Axum, SurrealDB (embedded RocksDB or in-memory), ammonia (HTML sanitization)
Document conversion	Markdown ↔ HTML, PDF via headless Chrome, Word via Pandoc

Layer	Technologies
AI/LLM	Multi-provider trait abstraction: Mock, OpenAI, OpenRouter, Anthropic; SSE streaming
Protocols	REST HTTP API + MCP over Streamable HTTP (JSON-RPC 2.0)

4. Features

- Base portal unifying all functions
- User management with OAuth2 (Google, GitHub, etc.) — deferred to the final phase
- File storage and management (Phase 4)
- Idea management and organization with a reusable Markdown editor (Phase 5)
- Data visualization and analytics
- AI/LLM integration — multi-provider, multi-model, multi-modal (text, image, video, audio) (Phase 6)
- HTML document editor with a shared conversion/export pathway (Phase 7)
- Export to PDF and MS Office (Word, Excel, PowerPoint)
- Markdown support throughout

5. The Knowledge Layer (Phase 11)

Baitler ships an agentic knowledge layer surfaced through **36 MCP tools** covering ideas, documents, files, folders, AI, export, and health. Highlights:

- **Projects** — group ideas, documents, and files via each item's `project_id`.
- **Knowledge graph** — a symmetric `kn_link` cross-type link graph with backlinks.
- **Full-text search** — BM25 `knowledge_search` across the base (analyzer + per-field SEARCH indexes).
- **Activity log** — an append-only provenance log; agent actions are attributed via the `x-Baitler-Agent` header.
- **Human-in-the-loop** — agent-authored ideas & documents default to `review="draft"` for approval; `review="published"` skips the queue.

The agent loop: **ORGANISE** (create projects, add items, link them) → **RETRIEVE** (search, then ground a reply with `ai_chat`) → **EXPORT** (turn documents into pdf/docx/markdown/html). *This very document was authored and published through that loop.*

6. Build & Run Commands

Workspace

- `cp .env.example .env` — create local config; fill in secrets/OAuth/LLM keys.
- `./scripts/dev.sh` — run the full dev stack; Ctrl-C tears all down.
- `./scripts/build.sh` — release build (cargo build --release + vite build).

Backend (from `backend/`)

- `cargo run` — start the API (`GET /health` , `GET /version`); defaults to a file-based embedded SurrealDB, no server or Docker needed.
- `cargo test` — unit + integration tests against an ephemeral in-memory DB.
- `cargo clippy --all-targets -- -D warnings` · `cargo fmt --all`

Frontend (from `frontend/`)

- `npm run dev` — Vite dev server on port 8100.
- `npm run build` — `tsc -b` + vite build · `npm run lint` · `npm run typecheck` · `npm test` (Vitest).

7. MCP Server

The `baitler-api` process serves MCP over Streamable HTTP at `POST /mcp` (in-process, reusing the same repos/DB). A companion binary `baitler-mcp` is a stdio ↔ HTTP bridge for clients that only launch stdio servers. Config: `MCP_ENABLED` (default true), `MCP_AUTH_TOKEN` (optional bearer). Binary blobs (PDF/DOCX/file reads) are returned Base64-encoded.

8. Engineering Notes

- LLM access is multi-provider behind an `LlmProvider` trait — built up front rather than coding against one vendor SDK.
 - Document conversion is a single shared pathway (`src/convert.rs`) rather than per-feature one-offs.
 - Auth is OAuth2-first (third-party identity providers), not username/password.
 - CI runs `fmt/clippy/test` for the backend and `lint/typecheck/build/test` for the frontend.
 - Toolchain versions are pinned: Node in `.nvmrc` , Rust in `rust-toolchain.toml` .
-

